

IIWM-PIPER Portal

```
import streamlit as st
import pandas as pd
import math
import base64

st.set_page_config(page_title="Piper Dashboard", layout="wide",
initial_sidebar_state="expanded")

st.markdown("""
    <style>
        .stApp { background: linear-gradient(135deg, #f5f7fa 0%, #c3cfe2
100%); }

        #MainMenu {visibility: hidden;}
        footer {visibility: hidden;}
        header {visibility: hidden;}
        .dash-card {
            background-color: #ffffff;
            border-radius: 16px;
            padding: 24px;
            box-shadow: 0 8px 16px rgba(0,0,0,0.05);
            margin-bottom: 24px;
        }
        div[data-testid="metric-container"] {
            background-color: #ffffff;
            border-radius: 12px;
            padding: 15px;
            box-shadow: 0 4px 6px rgba(0,0,0,0.04);
            border-left: 5px solid #1f77b4;
        }
        .main-title { font-family: 'Segoe UI', sans-serif; font-weight:
800; color: #2c3e50; margin-bottom: 0px; padding-top: 10px; }
        .sub-title { color: #7f8c8d; font-size: 16px; margin-bottom: 30px;
    }
    </style>
    """, unsafe_allow_html=True)
```

IIWM-PIPER Portal

```
PALETTE =[
    "#1f77b4", "#ff7f0e", "#2ca02c", "#d62728", "#9467bd",
    "#8c564b", "#e377c2", "#7f7f7f", "#bcbd22", "#17becf",
    "#393b79", "#637939", "#8c6d31", "#843c39", "#7b4173",
    "#3182bd", "#31a354", "#756bb1", "#636363", "#e6550d"
]

MARKERS =[
    "circle", "square", "triangle-up", "diamond", "triangle-down",
    "cross", "x", "triangle-left", "triangle-right", "pentagon"
]

EQ_WT = {
    "Ca": 20.04, "Mg": 12.15, "Na": 23.0, "K": 39.1,
    "HCO3": 61.0, "CO3": 30.0, "Cl": 35.45, "SO4": 48.03,
}

WIDTH, HEIGHT, SCALE, OX, OY = 1200, 820, 320, 100, 710
h = math.sqrt(3) / 2
side = 1.0
gap = 0.55
diamondShiftY = -0.28

L1, L2, L3 =[0.0, 0.0],[side, 0.0],[side / 2, h]
R1, R2, R3 =[side + gap, 0.0],[2 * side + gap, 0.0],[1.5 * side + gap, h]

D_left =[0.5 * side + gap / 2, h + gap / 2 + diamondShiftY]
D_right =[1.5 * side + gap / 2, h + gap / 2 + diamondShiftY]
D_top =[ (D_left[0] + D_right[0]) / 2, D_left[1] + h]
D_bottom =[ (D_left[0] + D_right[0]) / 2, D_left[1] - h]

def to_num(v):
    try:
        val = float(v)
```

IIWM-PIPER Portal

```
        return val if math.isfinite(val) else None
    except (ValueError, TypeError):
        return None

def get_val(row, keys):
    for k in keys:
        if k in row: return row[k]
    return None

def ternary_point(v1, v2, v3, p1, p2, p3):
    s = v1 + v2 + v3
    if s == 0: return[0, 0]
    return[(v1 * p1[0] + v2 * p2[0] + v3 * p3[0]) / s, (v1 * p1[1] + v2 *
p2[1] + v3 * p3[1]) / s]

def diamond_xy(ca_mg, so4_cl):
    u, v = ca_mg / 100.0, so4_cl / 100.0
    cx = (D_left[0] + D_right[0]) / 2.0
    half_width = (D_right[0] - D_left[0]) / 2.0
    y0 = D_bottom[1]
    return[cx + half_width * (v - u), y0 + h * (u + v)]

def map_point(p):
    return[OX + p[0] * SCALE, OY - p[1] * SCALE]

def render_b64_image(svg_string):
    b64 = base64.b64encode(svg_string.encode('utf-8')).decode('utf-8')
    return f"data:image/svg+xml;base64,{b64}"

def draw_line(points, stroke="#cbd5e1", sw=1):
    pts = " ".join([f"{map_point(p)[0]},{map_point(p)[1]}" for p in
points])
    return f"<polyline points='{pts}' fill='none' stroke='{stroke}' stroke-
width='{sw}' stroke-dasharray='4 4' />"

def triangle_grid(A, B, C, step=20):
```

IIWM-PIPER Portal

```
svg = []

for t in range(step, 100, step):

    for p1_args, p2_args in [((t, 100-t, 0), (t, 0, 100-t)), ((100-t, t,
0), (0, t, 100-t)), ((100-t, 0, t), (0, 100-t, t))]:

        p1, p2 = ternary_point(*p1_args, A, B, C),
ternary_point(*p2_args, A, B, C)

        svg.append(draw_line([p1, p2]))

return "\n".join(svg)


def diamond_grid(step=20):

    svg = []

    for p in range(step, 100, step):

        q = p / 100.0

        s1 = [(1 - q) * D_left[0] + q * D_bottom[0], (1 - q) * D_left[1] + q
* D_bottom[1]]

        e1 = [(1 - q) * D_top[0] + q * D_right[0], (1 - q) * D_top[1] + q *
D_right[1]]

        s2 = [(1 - q) * D_left[0] + q * D_top[0], (1 - q) * D_left[1] + q *
D_top[1]]

        e2 = [(1 - q) * D_bottom[0] + q * D_right[0], (1 - q) * D_bottom[1]
+ q * D_right[1]]

        svg.append(draw_line([s1, e1]))

        svg.append(draw_line([s2, e2]))

    return "\n".join(svg)


def poly_points(points):

    return " ".join([f"{map_point(p)[0]},{map_point(p)[1]}" for p in
points])


def diamond_background():

    midLT = [(D_left[0] + D_top[0]) / 2, (D_left[1] + D_top[1]) / 2]
    midTR = [(D_top[0] + D_right[0]) / 2, (D_top[1] + D_right[1]) / 2]
    midLB = [(D_left[0] + D_bottom[0]) / 2, (D_left[1] + D_bottom[1]) / 2]
    midRB = [(D_right[0] + D_bottom[0]) / 2, (D_right[1] + D_bottom[1]) /
2]

    midC = [(D_top[0] + D_bottom[0]) / 2, (D_top[1] + D_bottom[1]) / 2]

    return f"""
```

IIWM-PIPER Portal

```
<polygon points="{poly_points([midLT, D_top, midTR, midC])}"
fill="#bfe3c6" opacity="0.6" />

<polygon points="{poly_points([D_left, midLT, midC, midLB])}"
fill="#d5a7ec" opacity="0.6" />

<polygon points="{poly_points([midTR, D_right, midRB, midC])}"
fill="#f5b1b1" opacity="0.6" />

<polygon points="{poly_points([midLB, midC, midRB, D_bottom])}"
fill="#a9dcff" opacity="0.6" />

"""

def symbol_path(marker, x, y, size, color):

    s = size

    if marker == "square": return f'<rect x="{x - s/2}" y="{y - s/2}"
width="{s}" height="{s}" fill="{color}" stroke="#fff" stroke-width="0.5"/>'

    elif marker == "triangle-up": return f'<polygon points="{x},{y - s/2}
{x - s/2},{y + s/2} {x + s/2},{y + s/2}" fill="{color}" stroke="#fff"
stroke-width="0.5"/>'

    elif marker == "diamond": return f'<polygon points="{x},{y - s/2} {x -
s/2},{y} {x},{y + s/2} {x + s/2},{y}" fill="{color}" stroke="#fff" stroke-
width="0.5"/>'

    else: return f'<circle cx="{x}" cy="{y}" r="{s/2}" fill="{color}"
stroke="#fff" stroke-width="0.5"/>'

def render_text(point, text, align="middle", rotate=0, offset=(0,0),
font_size=16, color="#1e293b", weight="bold"):

    x, y = map_point(point)

    rot_str = f'transform="rotate({rotate} {x+offset[0]} {y+offset[1]})"'
    if rotate else ""

    return f'<text x="{x+offset[0]}" y="{y+offset[1]}" text-
anchor="{align}" font-size="{font_size}" font-weight="{weight}"
fill="{color}" {rot_str} font-family="sans-serif">{text}</text>'

def render_multiline_text(point, lines, font_size=11, fill="#64748b",
opacity=0.9):

    x, y = map_point(point)

    svg = []

    line_height = font_size * 1.3

    start_y = y - ((len(lines) - 1) * line_height) / 2 + (font_size / 3)

    for i, line in enumerate(lines):

        svg.append(f'<text x="{x}" y="{start_y + i*line_height}" text-
anchor="middle" font-size="{font_size}" font-weight="700" fill="{fill}"
```

IIWM-PIPER Portal

```
opacity="{opacity}" font-family="sans-serif" letter-spacing="0.5">{line}</text>')
```

```
    return "\n".join(svg)
```

```
with st.sidebar:
```

```
    st.markdown("<h2> Dashboard Settings</h2>", unsafe_allow_html=True)
```

```
    st.markdown("### 1. Data Upload")
```

```
    uploaded_file = st.file_uploader("Upload Excel File", type=["xlsx",  
"xls"])
```

```
    st.markdown("### 2. Plot Configuration")
```

```
    title = st.text_input("Plot Title", "Hydrochemical Piper Plot")
```

```
    point_size = st.slider("Marker Size", min_value=1, max_value=20,  
value=7)
```

```
    st.markdown("### 3. Data Grouping")
```

```
    group_placeholder = st.empty()
```

```
    label_placeholder = st.empty()
```

```
    st.markdown("### 4. Display Options")
```

```
    show_labels = st.checkbox("Show Sample Labels", False)
```

```
    show_facies = st.checkbox("Show Facies Text", True)
```

```
    show_scales = st.checkbox("Show Axis Scales (20-80)", True)
```

```
st.markdown("<h1 class='main-title'> PIPER ANALYSIS DASHBOARD</h1>",  
unsafe_allow_html=True)
```

```
st.markdown("<p class='sub-title'>ICAR -IIWM 2026</p>",  
unsafe_allow_html=True)
```

```
df, valid_rows, table_data, groups, style_map = pd.DataFrame(), [], [], [],  
{}
```

```
if uploaded_file is not None:
```

```
    try:
```

```
        df = pd.read_excel(uploaded_file)
```

```
        df.columns = [str(c).strip() for c in df.columns]
```

IIWM-PIPER Portal

```
fields = list(df.columns)

pref_fields=["District Name", "District", "Location", "Sample ID",
"Sample_Id", "Sample"]

default_idx = next((i for i, f in enumerate(fields) if f in
pref_fields), 0)

group_field = group_placeholder.selectbox("Group by Column",
fields, index=default_idx)

label_field = label_placeholder.selectbox("Label by Column",
fields, index=default_idx)

for idx, r in df.iterrows():

    ca = to_num(get_val(r, ["Ca ppm", "Ca"]))
    mg = to_num(get_val(r, ["Mg ppm", "Mg"]))
    na = to_num(get_val(r, ["Na ppm", "Na"]))
    k = to_num(get_val(r, ["K (ppm)", "K ppm", "K"])) or 0.0
    co3 = to_num(get_val(r, ["CO3"])) or 0.0
    hco3 = to_num(get_val(r, ["HCO3- ppm", "HCO3 ppm", "HCO3"]))
    cl = to_num(get_val(r, ["Cl ppm", "Cl"]))
    so4 = to_num(get_val(r, ["S ppm", "S", "SO4", "SO4 ppm"]))

    if None not in [ca, mg, na, hco3, cl, so4]:

        valid_r = {
            "group": str(r[group_field]).strip() if
pd.notna(r[group_field]) else "Unknown",
            "label": str(r[label_field]).strip() if
pd.notna(r[label_field]) else ""
        }

        Ca_meq, Mg_meq, Na_meq, K_meq = ca/EQ_WT["Ca"],
mg/EQ_WT["Mg"], na/EQ_WT["Na"], k/EQ_WT["K"]

        HCO3_meq, CO3_meq, Cl_meq, SO4_meq = hco3/EQ_WT["HCO3"],
co3/EQ_WT["CO3"], cl/EQ_WT["Cl"], so4/EQ_WT["SO4"]

        cat_sum, an_sum = (Ca_meq+Mg_meq+Na_meq+K_meq),
(HCO3_meq+CO3_meq+Cl_meq+SO4_meq)

        if cat_sum > 0 and an_sum > 0:
```

IIWM-PIPER Portal

```
Ca_pct, Mg_pct, NaK_pct = (Ca_meq/cat_sum)*100,
(Mg_meq/cat_sum)*100, ((Na_meq+K_meq)/cat_sum)*100

Cl_pct, SO4_pct, HCO3CO3_pct = (Cl_meq/an_sum)*100,
(SO4_meq/an_sum)*100, ((HCO3_meq+CO3_meq)/an_sum)*100

valid_r["cat"] = ternary_point(Ca_pct, NaK_pct, Mg_pct,
L1, L2, L3)

valid_r["an"] = ternary_point(HCO3CO3_pct, Cl_pct,
SO4_pct, R1, R2, R3)

valid_r["dia"] = diamond_xy(Ca_pct + Mg_pct, SO4_pct +
Cl_pct)

valid_rows.append(valid_r)

cat_f = "Calcium type" if Ca_pct > 50 else "Magnesium
type" if Mg_pct > 50 else "Sodium type" if NaK_pct > 50 else "No dominant"

an_f = "Bicarbonate type" if HCO3CO3_pct > 50 else
"Sulfate type" if SO4_pct > 50 else "Chloride type" if Cl_pct > 50 else "No
dominant"

if (Ca_pct+Mg_pct) >= 50 and HCO3CO3_pct >= 50: gw_type
= "Ca-Mg-HCO3"

elif NaK_pct > 50 and HCO3CO3_pct >= 50: gw_type = "Na-
HCO3"

elif NaK_pct > 50 and (SO4_pct+Cl_pct) > 50: gw_type =
"Na-Cl" if Cl_pct >= SO4_pct else "Na-SO4"

elif (Ca_pct+Mg_pct) >= 50 and (SO4_pct+Cl_pct) > 50:
gw_type = "Ca-Mg-Cl" if Cl_pct >= SO4_pct else "Ca-Mg-SO4"

else: gw_type = "Mixed"

table_data.append({"Group": valid_r["group"], "Sample
Label": valid_r["label"], "Cation facies": cat_f, "Anion facies": an_f,
"Groundwater type": gw_type})

groups = sorted(list(set(r["group"] for r in valid_rows)))

for i, g in enumerate(groups): style_map[g] = {"color": PALETTE[i %
len(PALETTE)], "marker": MARKERS[i % len(MARKERS)]}

m1, m2, m3 = st.columns(3)

m1.metric(label="Total Valid Samples", value=f"{len(valid_rows)}")

m2.metric(label="Total Groups Identified", value=f"{len(groups)}")
```


IIWM-PIPER Portal

```
m3.metric(label="Dominant Water Type",
value=pd.DataFrame(table_data)["Groundwater type"].mode()[0] if table_data
else "N/A")

except Exception as e:

    st.error(f"Error parsing file: {e}")

else:

    group_placeholder.selectbox("Group by Column", ["Upload file first"],
disabled=True)

    label_placeholder.selectbox("Label by Column", ["Upload file first"],
disabled=True)

    st.info("👋 Welcome! Please upload an Excel file from the sidebar to
generate your dashboard.")

if len(valid_rows) > 0:

    facies_svg = ""

    if show_facies:

        facies_svg += render_multiline_text([0.5, h * 0.65], ["MAGNESIUM",
"TYPE"])

        facies_svg += render_multiline_text([0.25, h * 0.16], ["CALCIUM",
"TYPE"])

        facies_svg += render_multiline_text([0.75, h * 0.16], ["SODIUM",
"TYPE"])

        facies_svg += render_multiline_text([0.5, h * 0.33], ["NO DOMINANT",
"TYPE"])

        facies_svg += render_multiline_text([R1[0] + 0.5, h *
0.65], ["SULFATE", "TYPE"])

        facies_svg += render_multiline_text([R1[0] + 0.25, h *
0.16], ["BICARBONATE", "TYPE"])

        facies_svg += render_multiline_text([R1[0] + 0.75, h * 0.16],
["CHLORIDE", "TYPE"])

        facies_svg += render_multiline_text([R1[0] + 0.5, h * 0.33], ["NO
DOMINANT", "TYPE"])

        facies_svg += render_multiline_text([D_top[0], D_top[1] -
0.43], ["CALCIUM MAGNESIUM", "SULFATE"])

        facies_svg += render_multiline_text([D_bottom[0], D_bottom[1] +
0.43], ["SODIUM", "BICARBONATE"])

        facies_svg += render_multiline_text([D_left[0] + 0.25,
D_left[1]], ["CALCIUM MAGNESIUM", "BICARBONATE"])
```

IIWM-PIPER Portal

```
    facies_svg += render_multiline_text([D_right[0] - 0.25,
D_right[1]],["SODIUM", "CHLORIDE"])

    scale_svg = ""

    if show_scales:

        for t in [20, 40, 60, 80]:

            scale_svg += render_text(ternary_point(t, 100-t, 0, L1, L2,
L3), str(t), offset=(0, 14), font_size=10, color="#475569",
weight="normal")

            scale_svg += render_text(ternary_point(100-t, 0, t, L1, L2,
L3), str(t), offset=(-8, 4), align="end", font_size=10, color="#475569",
weight="normal")

            scale_svg += render_text(ternary_point(0, t, 100-t, L1, L2,
L3), str(t), offset=(8, 4), align="start", font_size=10, color="#475569",
weight="normal")

            scale_svg += render_text(ternary_point(100-t, t, 0, R1, R2,
R3), str(t), offset=(0, 14), font_size=10, color="#475569",
weight="normal")

            scale_svg += render_text(ternary_point(t, 0, 100-t, R1, R2,
R3), str(t), offset=(-8, 4), align="end", font_size=10, color="#475569",
weight="normal")

            scale_svg += render_text(ternary_point(0, 100-t, t, R1, R2,
R3), str(t), offset=(8, 4), align="start", font_size=10, color="#475569",
weight="normal")

            scale_svg += render_text(diamond_xy(100, t), str(t), offset=(-
8, -4), align="end", font_size=10, color="#475569", weight="normal")

            scale_svg += render_text(diamond_xy(t, 100), str(t), offset=(8,
-4), align="start", font_size=10, color="#475569", weight="normal")

            scale_svg += render_text(diamond_xy(t, 0), str(t), offset=(-8,
10), align="end", font_size=10, color="#475569", weight="normal")

            scale_svg += render_text(diamond_xy(0, t), str(t), offset=(8,
10), align="start", font_size=10, color="#475569", weight="normal")

    svg_points = ""

    for r in valid_rows:

        grp = r["group"]

        c, m = style_map[grp]["color"], style_map[grp]["marker"]

        x1, y1 = map_point(r["cat"])

        x2, y2 = map_point(r["an"])
```

IIWM-PIPER Portal

```
x3, y3 = map_point(r["dia"])

svg_points += symbol_path(m, x1, y1, point_size, c)
svg_points += symbol_path(m, x2, y2, point_size, c)
svg_points += symbol_path(m, x3, y3, point_size, c)

if show_labels and r["label"]:
    offset = point_size / 2 + 4
    lbl = r["label"]

    svg_points += f'<text x="{x1+offset}" y="{y1+offset}" font-
size="10" fill="#475569" font-weight="bold" font-family="sans-
serif">{lbl}</text>'

    svg_points += f'<text x="{x2+offset}" y="{y2+offset}" font-
size="10" fill="#475569" font-weight="bold" font-family="sans-
serif">{lbl}</text>'

    svg_points += f'<text x="{x3+offset}" y="{y3+offset}" font-
size="10" fill="#475569" font-weight="bold" font-family="sans-
serif">{lbl}</text>'

svg_content = f"""

<svg width="{WIDTH}" height="{HEIGHT}" viewBox="0 0 {WIDTH} {HEIGHT}"
xmlns="http://www.w3.org/2000/svg">

    <text x="{WIDTH/2}" y="45" text-anchor="middle" font-size="32"
font-weight="800" fill="#1e293b" font-family="sans-serif">{title}</text>

    <polygon points="{poly_points([L1,[0.5, 0],[0.25, h / 2]])}"
fill="#fecaca" opacity="0.6" />

    <polygon points="{poly_points([[0.5, 0], L2,[0.75, h / 2]])}"
fill="#d9f99d" opacity="0.6" />

    <polygon points="{poly_points([[0.25, h / 2],[0.75, h / 2], L3]])}"
fill="#e9d5ff" opacity="0.6" />

    <polygon points="{poly_points([[0.25, h / 2],[0.75, h / 2],[0.5, h
/ 4]])}" fill="ffffff" opacity="0.9" />

    <polygon points="{poly_points([R1,[R1[0] + 0.5, 0], [R1[0] + 0.25,
h / 2]])}" fill="#bae6fd" opacity="0.6" />

    <polygon points="{poly_points([R1[0] + 0.5, 0], R2,[R1[0] + 0.75,
h / 2]])}" fill="#fef08a" opacity="0.6" />

    <polygon points="{poly_points([R1[0] + 0.25, h / 2], [R1[0] +
0.75, h / 2], R3]])}" fill="#a7f3d0" opacity="0.6" />
```

IIWM-PIPER Portal

```
<polygon points="{poly_points([[R1[0] + 0.25, h / 2], [R1[0] +
0.75, h / 2],[R1[0] + 0.5, h / 4]])}" fill="#ffffff" opacity="0.9" />

{diamond_background()}

{facies_svg}
{triangle_grid(L1, L2, L3)}
{triangle_grid(R1, R2, R3)}
{diamond_grid()}

{draw_line([L1, L2, L3, L1], "#475569", 1.8)}
{draw_line([R1, R2, R3, R1], "#475569", 1.8)}
{draw_line([D_left, D_top, D_right, D_bottom, D_left], "#475569",
1.8)}

{render_text([L1[0] + 0.50, -0.14], "Ca", font_size=18)}
{render_text([L3[0] - 0.50, h / 2], "Mg", rotate=-60,
font_size=18)}
{render_text([L2[0] - 0.06, 0.38], "Na+K", rotate=60,
align="middle", font_size=18)}

{render_text([R2[0] - 0.50, -0.14], "Cl", font_size=18)}
{render_text([R3[0] + 0.50, h / 2], "SO4", rotate=60,
font_size=18)}
{render_text([R1[0] + 0.06, 0.38], "CO3+HCO3", rotate=-60,
align="middle", font_size=18)}

{render_text([D_left[0] + 0.12, D_top[1] - 0.50], "SO4+Cl",
rotate=-60, align="middle", font_size=18)}
{render_text([D_right[0] - 0.12, D_top[1] - 0.50], "Ca+Mg",
rotate=60, align="middle", font_size=18)}

{scale_svg}
{svg_points}
</svg>
"""

st.markdown(f'''
```

IIWM-PIPER Portal

```
<div class="dash-card" style="display: flex; justify-content: center;
overflow-x: auto; padding-bottom: 5px; margin-bottom: 5px;">

    

</div>

'', unsafe_allow_html=True)

col_dl1, col_dl2, col_dl3 = st.columns([1, 2, 1])
with col_dl2:

    st.download_button(

        label="📄 Download Piper Diagram (High-Res SVG)",
        data=svg_content,
        file_name="Piper_Diagram.svg",
        mime="image/svg+xml",
        use_container_width=True

    )

st.markdown("<br>", unsafe_allow_html=True)

col_leg, col_tab = st.columns([1, 2])
with col_leg:

    st.markdown("<h3 style='color:#2c3e50;'>🔑 Legend</h3>",
unsafe_allow_html=True)

    legend_html = '<div class="dash-card" style="display: flex; flex-
direction: column; gap: 12px; height: 400px; overflow-y: auto;">'

    for g in groups:

        c, m = style_map[g]["color"], style_map[g]["marker"]

        icon_svg = f'<svg width="18" height="18" viewBox="0 0 18 18"
xmlns="http://www.w3.org/2000/svg">{symbol_path(m, 9, 9, 12, c)}</svg>'

        icon_b64 = render_b64_image(icon_svg)

        legend_html += f'<div style="display: flex; align-items:
center; gap: 10px; font-size: 15px; color: #334155;"><span>{g}</span></div>'

    st.markdown(legend_html + '</div>', unsafe_allow_html=True)

with col_tab:
```

IIWM-PIPER Portal

```
st.markdown("<h3 style='color:#2c3e50;*>Groundwater  
Facies</h3>", unsafe_allow_html=True)
```

```
st.dataframe(pd.DataFrame(table_data), use_container_width=True,  
hide_index=True, height=400)
```